Drift-Aware Streaming Forecasts of NYC 311 Demand with Automated Retraining

Himanshu Parida
paridahimanshu0610@tamu.edu

1 Problem Description

NYC 311 is the city's hotline for non-emergency issues. People use it to report noise, broken heat, missed trash pickup, potholes, and more. Each request goes to a city agency, such as the Department of Sanitation or the Housing Department. These agencies must schedule field crews and call-center staff ahead of time. Too few staff cause long delays for residents. Too many staff waste money.

To plan well, agencies need to know how many requests are coming. This project forecasts the number of 311 requests in the next hours or days. The forecast is split by complaint category and area. For example, it could predict heat complaints in the Bronx for tomorrow.

The challenge is that demand keeps changing. A model trained once becomes less accurate over time. So the project also builds a system that detects these changes and updates the model automatically.

Goal: Build a system that makes accurate forecasts and stays accurate as the data changes. The system will:

- forecast request counts and a 90th percentile "busy-day" estimate,
- detect data drift (covariate shift, label shift, and concept drift),
- retrain the model automatically when needed, and
- process data as a stream.

Dataset: *311 Service Requests from 2020 to Present* from NYC Open Data [1]. It has millions of records. Each record has a time, complaint type, agency, and location. We will add free weather and holiday data.

Constraints:

- The data is large, so we need big data tools.
- The source updates daily. We will replay history in time order to simulate a stream. We will also poll the API for new records.
- Tests must respect time. We train on the past and test on the future.
- Only open-source tools are used.

Metrics:

- *Forecast error:* WAPE (Weighted Absolute Percentage Error) and MASE (Mean Absolute Scaled Error). MASE compares the model to a simple baseline.
- *Busy-day forecast:* quantile loss and coverage.
- *Drift handling:* detection delay, false-alarm rate, and recovery time after a shock.
- *System:* retraining cost and forecast latency.

2 Motivation

311 requests change over time. Weather, policies, and events like COVID change what people report. A model trained once will slowly get worse. Many real systems face this problem.

A good forecast helps agencies plan crews and call centers. It can reduce wait times for residents. It can also lower costs. The same approach can be used in other cities and in other streaming settings, such as demand forecasting for retail or ride services.

3 Planned Techniques

Forecasting models:

- *Baselines:* seasonal naive and ETS. The main model must beat these.
- *Main model:* LightGBM. One global model learns from all series. It supports quantile outputs.
- *Sequence models:* LSTM and GRU. These neural networks can learn long-term patterns. We will train them with quantile loss for the busy-day forecast.
- *Online model:* River for incremental updates.

Features: Lag counts, rolling averages, calendar and holiday features, weather, and category and location. All features use only past data.

Drift detection: We will test input features with statistical tests such as KS and PSI. We will also watch the forecast error with ADWIN. We will check whether each alarm is followed by a real drop in accuracy.

Retraining: A new "challenger" model is trained on recent data. It replaces the current "champion" only if it performs better on a held-out recent period. We will compare four strategies:

- never retrain,
- scheduled retraining,
- drift-triggered retraining, and
- online learning.

Streaming pipeline: Apache Kafka, Spark Structured Streaming, and a Delta Lake or Iceberg lakehouse. MLflow tracks models. Airflow runs jobs. Evidently checks drift. FastAPI serves forecasts. Grafana shows a dashboard.

Pros and cons. LightGBM is fast to retrain and works well on tabular data. LSTM and GRU can learn sequence patterns, but they need more compute and can forget old patterns when updated on new data. Drift-triggered retraining may save compute, but it risks false alarms. The experiments will test these trade-offs.

4 Deliverables and Milestones

Deliverables:

- Working open-source code in a public repository.
- A streaming pipeline with replay and live-polling modes.
- A drift monitor and an automated retraining workflow.
- An experiment comparing the four retraining strategies.
- A dashboard, final report, and in-class presentation.

Evaluation: We will replay data from 2020 onward in time order. All strategies see the same shocks. We will report the metrics above over time. The project succeeds if the system beats the baseline, recovers faster after shifts, and needs fewer retrains than a fixed schedule.

Phase	Task
1	Data exploration and drift check
2	Baseline and LightGBM models
3	Data pipeline and storage
4	Streaming (replay and live polling)
5	Drift monitoring and retraining experiments
6	Serving, dashboard, final report

References

[1] NYC Open Data. 311 Service Requests from 2020 to Present. `https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9`